IAM-01: IAM Governance Foundations

> **Series:** IAM — IAM Administration | **Notebook:** 1 of 12 | **Created:** January 2026 | **Last Updated:** 11/11/2026

Building a Strong Foundation for Identity Management

Effective IAM governance is the cornerstone of enterprise security. This notebook establishes the framework for managing identities, groups, policies, and access controls in Dynatrace's Gen3 IAM system.

> **Migrating from Management Zones?** If you have existing Management Zones and need to migrate them to Gen3 Policies and Boundaries, see the **MZ2POL** series instead. This series is for greenfield IAM governance.

Table of Contents

1. The ABAC Model
2. Governance Roles and Responsibilities
3. Account vs Environment Permissions
4. Centralized vs Federated Models
5. Documentation and Change Management
6. Governance Assessment

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Gen3 IAM enabled
Permissions	Account Administrator or IAM Administrator
Knowledge	Basic understanding of identity concepts

1. Gen3 IAM Overview

Dynatrace Gen3 IAM replaces the legacy Management Zone-based access control with a modern **Attribute-Based Access Control (ABAC)** model. This provides fine-grained, scalable access management for enterprises.

Key Components

Component	Purpose	Scope
Policies	Define what actions users can perform	Account or Environment
Boundaries	Narrow what a bound policy grants	Account (applied per binding)
Record fields (e.g. `dt.security_context`)	Carry the attribute that policy conditions and boundaries match on	Set at the data source or in OpenPipeline
Buckets	Organize data by retention time, cost allocation or query optimization	Environment
Segments	Reusable DQL-based data filters	Environment
Groups	Collections of users with shared access	Account
Users	Individual identities (local or federated)	Account

Data Partitioning Options

Choose the right isolation mechanism for your needs:

Mechanism	What It Does	Reversible	Best For
Record-level permissions (`dt.security_context` in policy conditions or boundaries)	Grants access per record	✅ Yes	Team access separation — the primary mechanism
Buckets	Physical storage partition	❌ No	Retention time, cost allocation, query optimization
Segments	Query-time view filter — never restricts access	✅ Yes	Ad-hoc data views

> ⚠️ **Key Decision:** If you need **hard data isolation** between teams (e.g., Team A can never see Team B's data), start with **record-level permissions**: set `dt.security_context` on the data and scope each team's policy binding to its value. Partition into **buckets** when the data also needs its own retention time, cost allocation or query scope — Dynatrace states buckets are not the primary access mechanism.
>
> **Sources:** Partition data (DT docs) — *"They are not intended as the primary mechanism for securing or segmenting data access."*, *"Buckets are best used for organizing data by retention time, cost allocation, or query optimization."*; Permissions in Grail (DT docs) — *"set the `dt.security_context` field either at the data source or in the processing pipeline."*

Why Gen3?

The Gen3 model provides several advantages over legacy Management Zones:

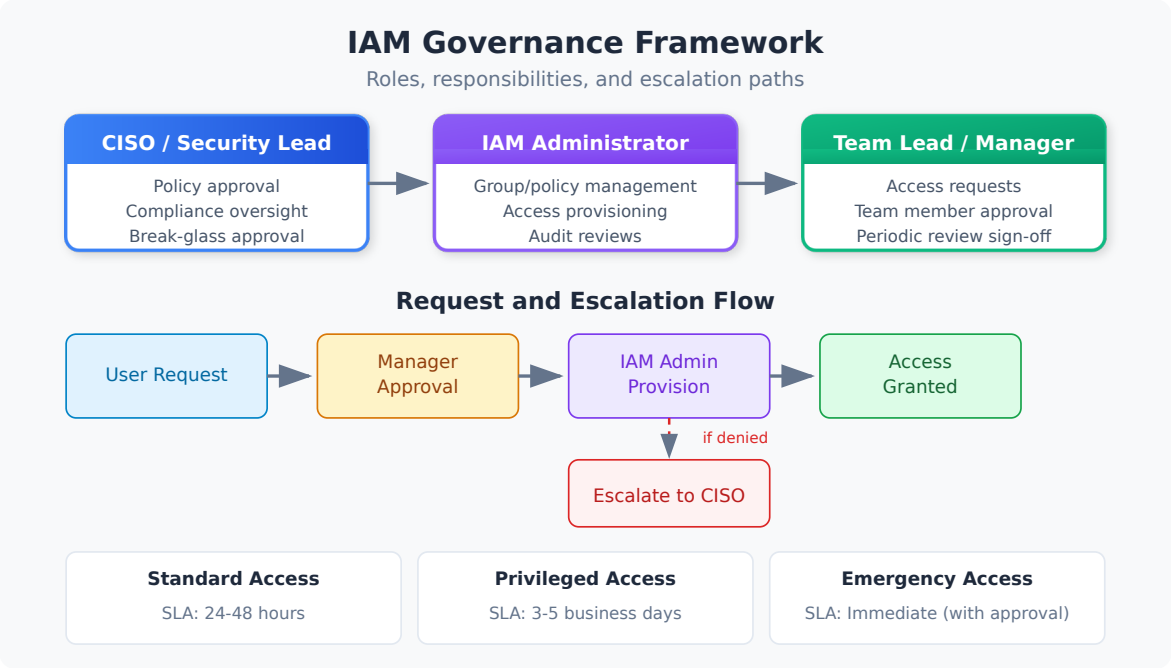
- **Scalability**: Policies and boundaries scale to thousands of entities
- **Flexibility**: Attribute-based rules adapt as your environment changes
- **Separation of Concerns**: Policies (actions) separate from boundaries (visibility)
- **Grail Integration**: Native support for Grail data lakehouse permissions

- **Record-level Permissions**: Grail access scoped per record, e.g. by `dt.security_context`

> **Note:** If you're migrating from Management Zones, see the **MZ2POL** series for migration guidance. This series assumes you're using Gen3 IAM.

2. The ABAC Model

Attribute-Based Access Control (ABAC) evaluates access requests based on attributes rather than static role assignments.



How Access is Evaluated

When a user attempts an action, Dynatrace evaluates:

1. **Identity**: Who is the user? What groups do they belong to?
2. **Policy**: Does any assigned policy grant this action?
3. **Boundary**: If the policy binding carries a boundary, does the record or object meet that boundary's condition for this permission?

Access is granted only if a policy allows the action — and, where a boundary is attached to that policy binding, the boundary's conditions for that permission are met. A boundary

narrows only the permissions its conditions apply to: a domain the boundary has no condition for is **left unrestricted**, not blocked. When more than one boundary applies to a policy, Dynatrace evaluates each boundary separately, so two boundaries on one binding (one per domain) each leave the other domain unconditional and **widen** access — put every domain's condition in one boundary. Boundaries don't apply to `DENY` statements. Segments then filter what the user *chooses* to view; they never restrict access.

> **Sources:** Policy boundaries (DT docs) — *"Only conditions added to service configuration will be applied to relative permissions."*, *"When more than one boundary applies to a policy, then effective statements are calculated for each boundary separately"*, *"Boundaries don't apply to `DENY` statements"*. **Derived:** the widening effect of two single-domain boundaries combines the first two quotes.

Policy Statement Structure

```
ALLOW &lt;service&gt;:&lt;resource&gt;:&lt;action&gt; [WHERE
&lt;condition&gt;];
```

Example:

```
ALLOW storage:buckets:read;
ALLOW storage:logs:read WHERE storage:dt.security_context = "team-a";
```

Every statement ends with `;`. A Grail table permission such as `storage:logs:read` also needs `storage:buckets:read`.

> **Sources:** IAM policy reference (DT docs) — `storage:buckets:read`: *"Required additionally to a table permission."*

Three Permission Domains

Domain	Controls	Examples
Environment	Classic environment permissions (`environment:roles:*`), scoped by `environment:management-zone`	Access environment, view data in a management zone

Domain	Controls	Examples
Storage	Grail records (logs, spans, metrics, events, entities), scoped by record fields such as `storage:dt.security_context`	Read logs, query spans
Settings	Configuration objects	Edit alerting rules

A boundary that has no condition for one of these domains leaves that domain **unrestricted** for the policy it is bound with.

> **Sources:** IAM policy reference (DT docs) — `environment:management-zone` : *"Applies the permission on the management zone level for the specified management zone."*; Policy boundaries (DT docs) — *"Only conditions added to service configuration will be applied to relative permissions."*

3. Governance Roles and Responsibilities

Effective IAM governance requires clear ownership and accountability. Define these roles within your organization.

Core IAM Roles

Role	Responsibilities	Typical Team
Account Administrator	Manage account-level settings, billing, environments	Platform Team
IAM Administrator	Manage groups, policies, boundaries, user access	Security/IAM Team
Environment Administrator	Manage environment-specific settings and integrations	Platform Team
Security Auditor	Review access logs, compliance reporting	Security/Compliance
Group Owner	Manage membership of specific groups	Application Teams

Separation of Duties

Implement checks and balances:

Action	Primary	Approver
Create new policy	IAM Admin	Security Lead
Add user to admin group	IAM Admin	Account Admin
Modify boundaries	IAM Admin	Environment Owner
Create service account	IAM Admin	Application Owner

RACI Matrix for IAM Operations

Activity	Account Admin	IAM Admin	Env Admin	Auditor
Policy creation	A	R	C	I
Group management	I	R	C	I
Access reviews	A	R	C	R
Compliance reporting	I	C	I	R
Break-glass procedures	R	R	I	I

R=Responsible, A=Accountable, C=Consulted, I=Informed

4. Account vs Environment Permissions

Understanding the two-tier permission model is critical for proper governance.

Account Level

Account-level permissions apply across ALL environments:

Permission	Grants
`account-viewer`	View account settings, environments

Permission	Grants
`account-company-info`	Manage account and company settings (also required to read the account audit log)
`account-user-management`	View and manage users, groups and policies

Environment Level

Environment access is granted by binding policies — Dynatrace's default policies or your own — to a group for one environment (or for the whole account):

Policy	Grants
Standard User (default policy)	Access the environment and run Dynatrace apps — **no Grail table reads**, and it does include document write and delete
All Grail data read access / **Read Logs**, **Read Spans**, … (default data-access policies)	Unconditional read of all Grail tables, or of one table and its buckets — pair with Standard User
Pro User (default policy)	Standard, plus build, deploy and run apps and workflows
Admin User (default policy)	Administrative access across all platform services
Custom policies	Fine-grained access per your design

Best Practice: Least Privilege

```
Account Level → Minimal, highly restricted
Environment Level → Scoped to team/function need
```

Example Structure:

User Type	Account Permission	Environment Permission
Platform Admin	account-user-management	Admin User (all)

User Type	Account Permission	Environment Permission
App Team Lead	account-viewer	Pro User (their env)
Developer	None	Standard User + custom policy (scoped reads + limited write)
Auditor	account-viewer	Standard User + a data-access policy (e.g. Read Logs, or All Grail data read access)

> **Standard User is not a read-only policy.** It grants no Grail table reads, so on its own it shows no logs, spans or metrics; and it does grant `document:documents:write` and `document:documents:delete`. Pair it with a data-access policy (or a custom `storage:*:read` policy) for read access.
>
> **Sources:** dynatrace_iam_group (Dynatrace GitHub) — *"Possible values: `account-company-info`, `account-user-management`, `account-viewer`"*; Default policies (DT docs) — Standard User: *"Grants access to Dynatrace environment and run Dynatrace Apps."*; All Grail data read access: *"Grants unconditional access to ALL Grail tables and to ALL Grail buckets."*

5. Centralized vs Federated Models

Choose the governance model that fits your organization's structure.

Centralized Model

A single team controls all IAM operations.

Characteristics:
- Central IAM team manages all policies, groups, boundaries
- Consistent standards across the organization
- Single point of control and audit

Best For:
- Regulated industries (finance, healthcare)
- Organizations requiring strict compliance
- Smaller deployments (< 500 users)

Implementation:

```
Central IAM Team → All policy/group/boundary management
App Teams → Request access via tickets
```

Federated Model

Distributed ownership with central oversight.

Characteristics:
- Business units manage their own groups/boundaries
- Central team sets standards and audits
- Faster response to team needs

Best For:
- Large enterprises with autonomous teams
- Organizations with diverse tech stacks
- DevOps/platform engineering models

Implementation:

```
Central IAM Team → Policies, standards, audit
Group Owners → Group membership management
Environment Owners → Environment-specific boundaries
```

Hybrid Model

Most enterprises adopt a hybrid approach:

Area	Ownership
Account-level policies	Central IAM team
Environment policies	Central IAM team
Group membership	Delegated to group owners
Boundary definitions	Collaborative (central + env owners)

6. Documentation and Change Management

Governance requires documentation and controlled changes.

Required Documentation

Document	Content	Review Frequency
Access Policy	Who can access what	Annual
Group Catalog	All groups, owners, purposes	Quarterly
Policy Inventory	All policies with descriptions	Quarterly
Boundary Definitions	Boundary rules and rationale	Quarterly
Onboarding Guide	New user access process	As needed
Offboarding Checklist	Access revocation steps	As needed

Change Management Process

All IAM changes should follow a controlled process:

1. **Request**: Ticket with business justification
2. **Review**: IAM admin reviews against policy
3. **Approval**: Required approver signs off
4. **Implementation**: Change made with audit trail
5. **Verification**: Requestor confirms access works
6. **Documentation**: Update catalogs/inventories

GitOps for IAM

Consider version-controlling your IAM configuration:

```
# account/iam.yaml — Monaco account configuration
policies:
  - name: team-a-log-reader
    id: team-a-log-reader
    level:
      type: account
    description: Read Team A logs
    policy: |-
      ALLOW storage:buckets:read;
      ALLOW storage:logs:read WHERE storage:dt.security_context = "team-a";
```

Monaco deploys account resources with its dedicated account commands — `monaco deploy` ignores them. See **IAM-04** for the full policy, boundary and group format.

> **Sources:** Monaco account configuration (DT docs) — *"existing commands like `monaco deploy` ignore any account configuration that may be defined in a manifest file"*.

Benefits:
- Full audit trail via git history
- Peer review before changes
- Rollback capability
- Documentation as code

See **AUTOM-03: Monaco** for configuration-as-code implementation.

7. Governance Assessment

Use these queries to assess your current IAM governance posture.

> **Classic and Smartscape service coverage are not the same number.** A Smartscape `SERVICE` node holds `dt.security_context` as an array (`[]` when unset), and its value *"comes from the resource level, so from each deployed instance"*; one service can aggregate instances with different contexts, so the value can change between observations. Run the Smartscape form given in the query comments next to the classic query, and look into any difference before you rely on either.

> **Sources:** Set up Grail permissions for Services (DT docs) — *"A single service entity can aggregate instances with different dt.security_context values, which is why the value on the service entity can change between observations"*.

```
// Count entities by security context - see how data is partitioned
fetch dt.entity.service
| summarize
    total = count(),
    withContext = countIf(isNotNull(dt.security_context)),
    missingContext = countIf(isNull(dt.security_context))
| fieldsAdd coveragePercent = round(100.0 * withContext / total, decimals: 2)

// Smartscape note (dt.entity.* is deprecated but still functional):
dt.security_context exists
// on Smartscape nodes but as an ARRAY (empty [] when unset, not null), so
isNull / isNotNull
// and by:{dt.security_context} do not carry over — a Smartscape rewrite would
miscount coverage.
// Keep the classic query above, and also run the documented Smartscape form,
which tests the array:
//   smartscapeNodes "SERVICE", from:-7d | summarize {total = count(),
withContext = countIf(dt.security_context != array())}
// The two can disagree: on the validation tenant (11/11/2026) the classic
query found a context on
// 58 of 58 services, Smartscape on 7 of 58. A Smartscape service takes its
value from its deployed
// instances ("from the resource level"), so they measure different things —
explain the gap before
// trusting either number.
```

```
// An empty result here is the HEALTHY answer, not a broken query (verified
08/12/2026: 0 of 37
// services lacked a security context on the validation tenant). Rows
appearing means governance
// gaps exist. Do not "fix" this cell because it returns nothing — confirm
with:
//   fetch dt.entity.service, from:-7d | summarize {total = count(),
no_context = countIf(isNull(dt.security_context))}
// List services without security context (governance gaps)
fetch dt.entity.service, from:-7d
| filter isNull(dt.security_context)
| fields entity.name, tags
| sort entity.name
| limit 50

// Smartscape note (dt.entity.* is deprecated but still functional):
dt.security_context exists
// on Smartscape nodes but as an ARRAY (empty [] when unset, not null), so
isNull / isNotNull
// and by:{dt.security_context} do not carry over — a Smartscape rewrite would
miscount coverage.
// Keep the classic query above, and also run the documented Smartscape form,
which tests the array:
//   smartscapeNodes "SERVICE", from:-7d | filter dt.security_context ==
array() | fields name
// The two can disagree: on the validation tenant (11/11/2026) the classic
query found a context on
// 58 of 58 services, Smartscape on 7 of 58. A Smartscape service takes its
value from its deployed
// instances ("from the resource level"), so they measure different things —
explain the gap before
// trusting either number.
// (entity tags are also not a flat "tags" field on Smartscape — resolve via
getNodeField.)
```

```
// Review security context distribution
fetch dt.entity.service
| filter isNotNull(dt.security_context)
| summarize serviceCount = count(), by:{dt.security_context}
| sort serviceCount desc
| limit 20

// Smartscape note (dt.entity.* is deprecated but still functional):
dt.security_context exists
// on Smartscape nodes but as an ARRAY (empty [] when unset, not null), so
isNull / isNotNull
// and by:{dt.security_context} do not carry over — a Smartscape rewrite would
miscount coverage.
// Keep the classic query above.
```

```
// Check host security context coverage
fetch dt.entity.host
| summarize
    total = count(),
    withContext = countIf(isNotNull(dt.security_context)),
    missingContext = countIf(isNull(dt.security_context))
| fieldsAdd coveragePercent = round(100.0 * withContext / total, decimals: 2)

// Smartscape note (dt.entity.* is deprecated but still functional):
dt.security_context exists
// on Smartscape nodes but as an ARRAY (empty [] when unset, not null), so
isNull / isNotNull
// and by:{dt.security_context} do not carry over — a Smartscape rewrite would
miscount coverage.
// Keep the classic query above.
```

Interpreting Results

Metric	Target	Action if Below
Security context coverage	> 95%	Review tagging strategy
Orphaned entities	< 5%	Assign ownership
Context distribution	Balanced	Review boundary design

8. Next Steps

With governance foundations in place, proceed to implement the specific components:

Recommended Path

1. **IAM-02: SSO and Authentication** - Configure SSO before users and groups
2. **IAM-03: Group Architecture and Design** - Design your group hierarchy
3. **IAM-04: Policy Authoring and Management** - Create custom policies
4. **IAM-05: Boundary Design Patterns** - Implement data boundaries

Bonus Workshop

> **IAM-11: Policy Persona Workshop** — A hands-on exercise for mapping organizational roles to Dynatrace IAM policies using persona-based design. Recommended after completing the core series.

Governance Checklist

Before moving on, ensure you have:

- [ ] Identified IAM roles and owners in your organization
- [ ] Chosen centralized, federated, or hybrid model
- [ ] Documented your access policy framework
- [ ] Established a change management process
- [ ] Assessed current security context coverage

Summary

In this notebook, you learned:

- The Gen3 IAM model and its components (policies, boundaries, segments)
- How ABAC evaluates access requests
- Core governance roles and their responsibilities

- The difference between account and environment permissions
- Centralized vs federated governance models
- Documentation and change management requirements

References

- IAM Overview
- Manage User Permissions with Policies
- Account Management

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.